



中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类

中山大学计算机学院



主讲人：万海



wanhai@mail.sysu.edu.cn

目录

CONTENTS

01

动态对象与指针

02

=delete 和 =default

03

对象成员初始化

04

对象的内存布局

05

拷贝构造函数

06

深入 const



review: C/C++程序内存布局

C++程序的内存格局通常分为四个区:

全局数据区(data area)

代码区(code area)

栈区(stack area)

堆区(heap area)(即自由存储区)。

全局数据区存放全局变量，静态数据和常量。

所有类成员函数和非成员函数代码存放在代码区；

为运行函数而分配的局部变量、函数参数、返回数据、返回地址等存放在栈区；

余下的空间都被称为堆区。



对象的内存布局

在类的定义时，类的成员函数被放在代码区。

类的静态成员变量在全局数据区。

非静态成员变量在类的实例内，实例在栈区或者堆区。

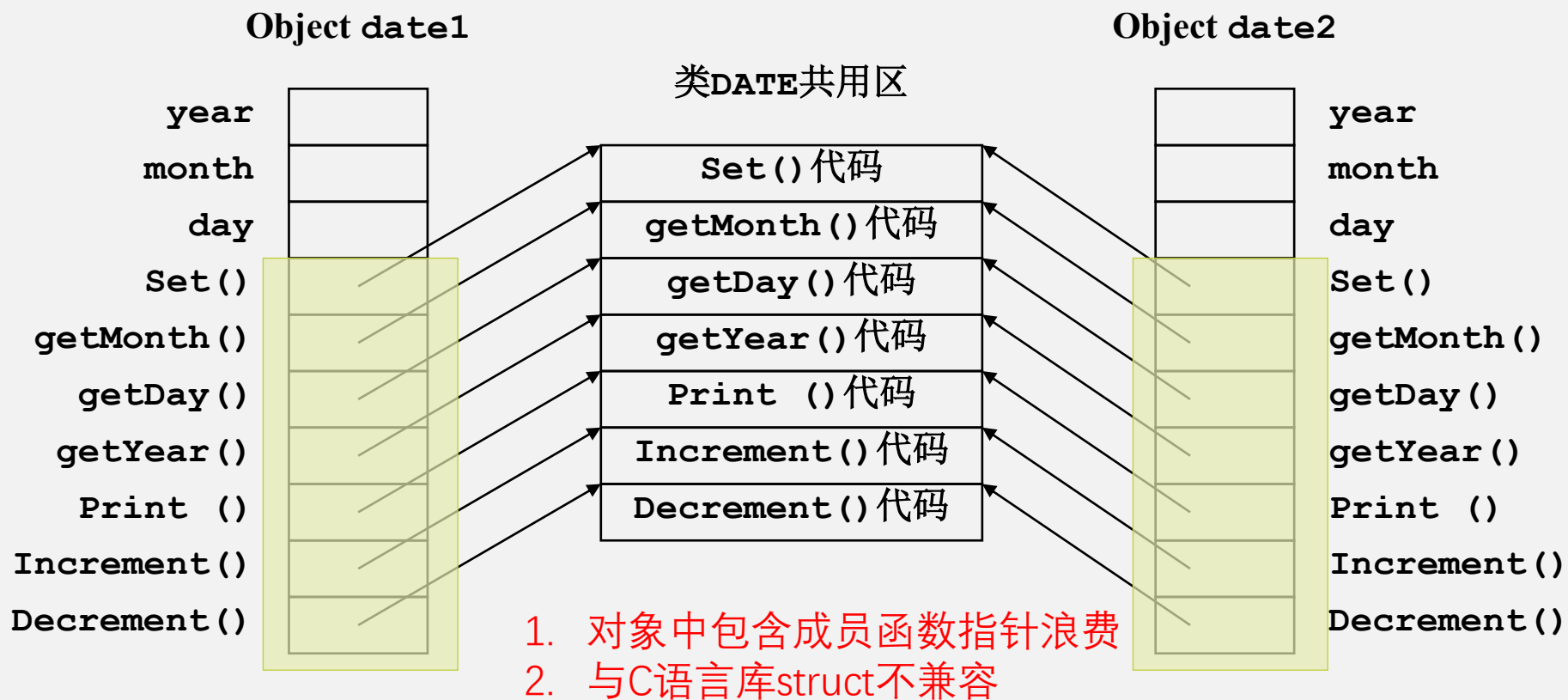
虚函数指针、虚基类指针在类的实例内，实例在栈区或者堆区。

类的实例如果是定义的类型变量，则在栈内存区，

如果是new出来的类指针，则在堆内存区，同时引用会保存在栈里。



对象的内存布局?



C++对象的内存布局与静态联编技术

```
DATE date1();  
DATE *date2 = new DATE();  
  
date1.Increment();  
date2->print();
```

C++如何翻译这段代码呢?

1. 在栈上按非静态数据结构分配空间，并调用构造函数初始化
2. 在堆上做类似操作
3. 将 `date1.Increment(...)` 翻译为 `DATE::Increment(&date1, ...)`
4. 类似 `DATE::Print(date2, ...)`

对象方法静态联编：指在编译阶段，就能直接使用代码段函数地址调用动态对象的方法。该方法仅需要向非静态成员函数传送this指针，即可用静态函数调用实现动态调用效果。

DATE date1

year	
month	
day	

DATE date2

year	
month	
day	

类DATE的代码段

Set () 代码
getMonth () 代码
getDay () 代码
getYear () 代码
Print () 代码
Increment () 代码
Decrement () 代码

静态联编的优势：

1. 对象布局与C结构内存布局一致，使得内存中对象便于与其他语言程序库兼容
2. 高效率，高性能



拷贝构造函数 - 概念

拷贝构造函数：在定义语句中用同类型的对象初始化另一个对象。

//假定C为已定义的类，则

C obja; //调用C的 (1) 无参构造函数，如无(1)(2)构造函数则用默认构造函数

C obja(1,2) //调用C的有参 (2) 普通构造函数

/*调用C的 (3) 拷贝构造函数用对象obja初始化对象objb。如果有为C类明确定义拷贝构造函数，将调用这个拷贝构造函数；如果没有为C类定义拷贝构造函数，将调用默认拷贝构造函数。*/

C objb(obja); //或

C objb = obja ; //两者等价（注意：不是赋值运算）



拷贝构造函数 - 语法

用**本类型对象引用**作第一形式参数的构造函数。

- 该参数传递方式为**对象引用**，避免在函数调用过程中生成形参副本
- 该形参声明为**const**，以确保在拷贝构造函数中不修改实参的值

C::C(const C& obj);

例如： **COMPLEX(const COMPLEX& other);**

注： C::C(C& obj) 不用 const 形式往往出现在旧的文档中。

C::C(const C& obj, ...) 比较少见（超纲）

C::C(C&& obj) 形式称为移动构造函数（超纲）



拷贝构造函数：要点

形参类型为该类类型本身且参数传递方式为按引用传递。

用一个已存在的该类对象初始化新创建的对象。

每个类都**必须有**拷贝构造函数：

- 用户可根据自己的需要显式定义拷贝构造函数。
- 若用户未提供，则该类使用由系统提供的**缺省拷贝构造函数（可用=default）**，也**可用 =delete 弃置该函数**。
- 缺省拷贝构造函数**按照初始化顺序**，对对象的各基类和**非静态成员进行完整的逐成员复制**，完成新对象的初始化。即逐一调用成员的拷贝构造，如果成员是基础类型，则复制值（赋值）。



隐式调用复制构造 (1) : 对象作为函数值参

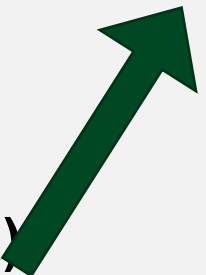
除了用 声明 `C x(c)` 或 `new C(c)` 会触发复制构造外，
将一个对象作为实参，以按值调用方式传递给被调函数的形参对象。

假定 **C** 为已定义的类，**obja** 为 **C** 类对象，且

```
void fun(C temp)
{
    ...
}
```

则

`fun(obja)`



用 **obja** 来初始化 **temp**。如果有为 **C** 类明确定义拷贝构造函数，将调用这个拷贝构造函数；如果没有为 **C** 类定义拷贝构造函数，将调用缺省的拷贝构造函数。

// **obja** 传递给 **fun** 函数，创建形参对象 **temp** 时，调用 **C** 的拷贝构造函数用对象 **obja** 初始化对象 **temp**，**temp** 生存期结束被撤销时，调用析构函数



隐式调用复制构造 (2) : 对象作为值从函数返回

生成一个临时对象作为函数的返回结果：当函数返回一对象时，系统将自动创建一个临时对象来保存函数的返回值。创建此临时对象时调用拷贝构造函数，当函数调用表达式结束后，撤销该临时对象时，调用析构函数。

```
C fun2()
```

//假定C为已定义的类，且

```
{
```

```
    C t;
```

```
    ...
```

```
    return t ;
```

```
}
```

则：

```
b = fun2( );
```

本页内容过时，见下页！

t --> 临时对象 --> b

拷贝构造函数

赋值运算符



对象作为值返回的编译优化

```
C func()  
{  
    C tmp;  
    ...  
    return tmp;  
}
```

但现在的gcc/g++不这么处理，会做一个优化。在C函数里有个tmp变量，离开func时不撤销这个对象，而是让new C和这个对象关联起来。也就是说tmp的地址和new C是一样的。



调用拷贝构造：例子

```
//例子: copy_constructing
class FOO {
public:
    FOO(int i)
    { cout << "Constructing.\n";
      member = i;
    }
    FOO(const FOO& other)
    { cout << "Copy constructing.\n";
      member = other.member;
    }
    ~FOO()
    { cout << "Destructing.\n"; }
    int get()
    { return member; }

private:
    int member;
};
```

```
void display ( FOO obj )
{
    cout << obj.get() << "\n";
}

FOO get_foo( )
{
    int value;

    cout<< "Input an integer:";
    cin>>value;
    FOO obj(value);

    return obj;
}
```



例子

```
int main() {  
    FOO obj1(15);          // 调用一般的构造函数  
  
    FOO obj2 = obj1;       // 调用拷贝构造函数  
  
    display(obj2);         // 调用拷贝构造函数用实参obj2初始化形参obj  
                           // 函数结束释放对象obj2  
  
    obj2=get_foo();        // 不会调用拷贝构造函数创建作为返回结果的临时对象  
                           // 调用一般构造函数，赋值后释放临时对象  
  
    display(obj2);         // ...  
  
    return 0;              //按声明自动变量逆顺序释放 obj2,obj1  
}
```



例子

运行结果分析

Constructing 15

Copy constructing 15

Copy constructing 15

15

Destructing.

Input an integer:7

Constructing 7

Destructing.

Copy constructing 7

7

Destructing.

Destructing.

Destructing.



复制策略：拷贝构造函数自定义

1. 对于不含指针成员类，使用系统提供（编译器合成）的**默认拷贝构造函数**即可。
2. 缺省拷贝构造函数使用**浅复制策略**，有时不满足**含指针数据成员**的类需要。
3. 含指针成员类通常应重写以下内容：
 - **构造函数**（及**拷贝构造函数**）中分配内存，**深复制策略**
 - **= 操作**重写，完成对象**深复制策略**
 - **析构函数**中释放内存

浅复制策略只按声明顺序复制成员的值，不复制指针指向的对象实体。它导致新旧对象成员指针指向同一块内存。**深复制策略**要求成员指针指向的对象也要复制，新对象跟原对象的成员指针不会指向同一块内存，修改新对象不会改到原对象。



浅拷贝问题：例子

```
//例子: shallow_copy_problems
class NAME {
public:
    NAME();
    ~NAME();
    void show();
    void set(const char* s);
private:
    char* str;
};

NAME::NAME():str(nullptr){
    cout << "Constructing.\n";
}

NAME::~~NAME() {
    cout << "Destructing.\n";
    if (str != nullptr)
        delete []str;
}
```

```
void NAME::show() {
    cout << str << "\n";
}

void NAME::set(const char* s) {
    if (str!=NULL)
        delete []str;

    str = new char[strlen(s) + 1];

    if (str!=NULL)
        strcpy(str, s);
}
```

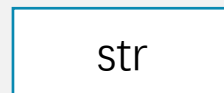


浅拷贝：问题分析（1）

```
void Print(NAME name) {  
    //②  
    name.show();  
}  
  
int main() {  
    NAME *myname = new NAME;  
    myname->set("Leetcode");  
    //①  
    Print(*myname);  
    //③  
    myname->show();  
    delete myname;  
}
```

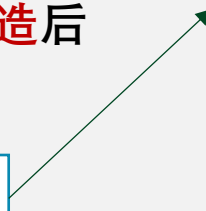
//①

*myname



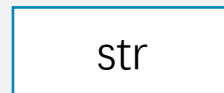
//② 调用默认拷贝构造后

name



//③ name对象析构，销毁 name.str 指向的对象！

*myname



// myname->show() 使用非法指针

// delete myname 二次 delete 相同地址



定义新拷贝构造函数

修改：加入拷贝构造函数的定义

```
NAME::NAME(const NAME& other){  
    cout << "Copy Constructing.\n";  
    if (other.str == nullptr) {  
        str = nullptr;  
        return;  
    }  
    str = new char[strlen(other.str) + 1];  
    strcpy(str, other.str);  
}  
//deepcopy_copy_construction.cpp
```

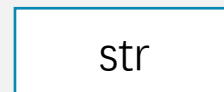


深拷贝策略：分析

```
void Print(NAME name) {  
    //②  
    name.show();  
}  
  
int main() {  
    NAME *myname = new NAME;  
    myname->set("Leetcode");  
    //①  
    Print(*myname);  
    //③  
    myname->show();  
    delete myname;  
}
```

//①

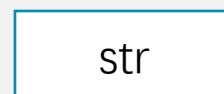
*myname



Leetcode\0

//② 调用**新拷贝构造**后

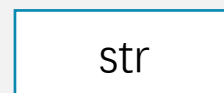
name



Leetcode\0

//③ name对象析构，销毁 name.str 指向的对象！

*myname



Leetcode\0

// myname->show() 正常输出
// delete myname 析构正常输出

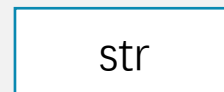


依然不完美的深拷贝：问题分析（2）

```
//deepcopy_without_assignment_op.cpp
int main() {
    NAME *myname = new NAME;
    myname->set("Leetcode");
    //①
    NAME *yourname = new NAME;
    *yourname = *myname;
    //②
    yourname->show();
    delete yourname;
    //③
    myname->show();
    delete myname;
}
```

//①

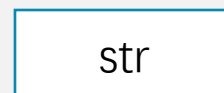
*myname



Leetcode\0

//② 调用默认赋值操作后

*yourname



//③ yourname对象析构，销毁 str 指向的对象！

*myname



//? //

// myname->show() 使用非法指针

// delete myname 二次 delete 相同地址



预览：深拷贝策略实现

```
//修改：加入赋值运算符重载函数的定义。 deepcopy_all.cpp
NAME& NAME::operator=(const NAME& other) {
    cout << "= operator.\n";
    if (this==&other) return *this;      //不能自身赋值
    if (str != NULL) delete[] str;      //释放对象资源

    if (other.str == NULL) {             //不需要资源
        str = NULL;
        return *this;
    }

    str = new char[strlen(other.str) + 1]; //分配对象新资源
    if (str != NULL)
        strcpy(str, other.str);          //复制资源

    return *this;
}
```



review: C语言 const

- const 常量 (相当于c++的 constexpr, 即编译期常数)
 - `const int ARR_SIZE = 10;`
 - `const float PI = 3.1415926;`
- const 与指针
 - `const int **p;` // 不能修改 **p, (*p)[]
 - `int *const *p;` // 不能修改 *p, p[]
 - `int ** const p;` // 不能修改 p
- const 与函数
 - `int foo(const int i);` //不能修改函数参数
 - `const int* foo();` //返回 const 指针, 必须强制类型转换赋值给非 const 指针



const 引用

- const 引用右值对象（为右值对象开辟临时空间，并赋予别名）
 - `const int &size = 15;`
 - `const int &size = i+7;`
- const 引用左值对象（直接赋予别名）
 - `const int &size = i;` //size 是 i 的别名
- const 引用隐式转型左值对象（开辟临时空间，并赋予别名）
 - `const int &size = pi;`
- const 引用作为函数参数和返回值
 - 作为函数形式参数，实参包含是右值、是左值、被隐式转换三种情况
 - 作为返回值，是返回临时对象的别名



const 引用 - 普通引用 (1)

```
// const_ref.cpp
int const_ref_with_rvalue() {
    // const引用右值
    const int &c = 15;
    //编译器会给常量15一个临时空间, 并赋予别名
    //等价: int temp=rValue; const int &c=temp;
    //    int &d=15; //error: invalid
    initialization of non-const reference of type
    'int&' from an rvalue of type 'int'
    int* p = (int *)&c; // must casting
    explicitly. p 指向临时空间
    *p = 10;
    cout << c << endl;
}
```

```
const_ref_with_rvalue...
10
const_ref_without_casting...
a=11,b=11
const_ref_with_casting...
a=3,b=11.000000
```

```
// const_ref.cpp
int const_ref_without_casting() {
    int b = 10;
    const int &a = b;
    //    a = 12; //error: assignment of read-
    only reference
    b = 11;
    printf("a=%d,b=%d\n", a, b);
}

int const_ref_with_casting() {
    double b = 3.14;
    const int &a = b; //等于: int temp = b;
    const int &a = temp
    b = 11;
    printf("a=%d,b=%lf\n", a, b);
}
```



const 引用 - 普通引用 (2)

```
// const_ref.cpp
int const_ref_in_para(const string& s) {
    cout << s << endl;
}

int no_const_ref_in_para(string& s) {
    cout << s << endl;
}

int const_ref_in_para(const int& s) {
    cout << s << endl;
}

int no_const_ref_in_para(int& s) {
    cout << s << endl;
}
```

```
// const_ref.cpp
int main() {
    cout << "const_ref_in_para..." << endl;
    const_ref_in_para("I love c++"); //equal
to:
    // string s_temp="I...";
    // const_ref_in_para(string(s_temp));

    // no_const_ref_in_para("I love c++");
    //error:...

    const_ref_in_para(3.14);
    // no_const_ref_in_para(3.14); //error
}
```

```
const_ref_in_para...
I love c++
3
```



const 引用 - 右值引用 (超纲)

const 限定标识符是常量, 则该符号是右值。

- 不能取地址
- 可以通过显式转型获取右值的指针
- 不能初始化左值引用
- 可以隐式转型右值为常引用
- 如该常量的右值需要修改, 则必要右值引用 (T&&) (超纲: 课程不要求右值引用相关语法)

```
// const_ref_rvalue.cpp
int main()
{
    const double pi = 3.14; //编译没有分配空间, pi是右值
    // double *r_pi = &pi; //语法错误?
    double *p_pi = (double *)&pi; //equal to
    // double temp = pi; double *p_pi = &temp;
    *p_pi = 4.0;
    //请解释输出结果
    printf("%lf,%lf\n", *p_pi, pi);

    // double &r_pi = pi; //语法错误?
    const double &r_pi = pi; //引用了pi的temp, 但无法修改
    // r_pi = 4.0; //语法错误?
    double &&r_pi = (double)pi; // (右值) 引用了pi的temp, 但可修改
    // double &&t_pi = *p_pi; //不能绑定左值到右值引用
    rr_pi = 5.0;
    //请解释输出结果
    printf("%lf,%lf,%lf,%lf\n", *p_pi, r_pi, rr_pi, pi);
}
```

本段代码仅为展示右值引用的原理。即右值引用是右值的临时空间, 并可以修改。
注意: cpp reference 明确指出, 修改 const 对象行为是未定义

- 常量对象是以下对象之一:
 - 类型具有 const 限定的对象
 - 常量对象的不可变子对象

这种对象不能被修改: 直接尝试这么做是编译时错误, 而间接尝试这么做 (例如通过到非常量类型的引用或指针修改常量对象) 的行为未定义。



const 成员（函数成员与数据成员）

```
// const_method.cpp
class Test {
public:
    void Test1(int _a)const { //常方法 其中this指针由Test* const--> const Test*const
        std::cout<<"Test()const"<<std::endl;
        a = _a; //error 常方法不能修改普通成员变量的值
        int d = _a; //可以访问a, 但不能修改a的值
        int e = c; // const函数访问const成员变量
    }
    void Test2() {
        int mf = c; //普通函数访问const成员变量
        std::cout<< "Test2(int,int)" <<std::endl; //输出该函数
    }
    Test(int _a,int _b,int _c):a(_a),b(_b),c(_c){ //构造函数(常成员由初始器列表进行初始化)
        std::cout<< "Test(int)" <<std::endl; //输出该函数
    }
private:
    int a, b;
    const int c; //C++中常非静态成员必须由默认初始器或初始器列表初始化
};
```



练习：Stack 的实现

阅读 stack.cpp 关于 栈类 的实现，回答以下问题：

- 注释去除拷贝构造函数，Stack(const Stack& other)
 - 运行程序并观察结果
- 请使用深拷贝策略正确完成拷贝构造函数
 - 如何复制链表？
 - 释放链表可以直接调用 clear 成员函数吗？
- 总结对象深拷贝策略的编程
 - 拷贝构造函数深拷贝策略的编程
 - 赋值操作深拷贝策略的编程
 - 析构函数的编程



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院